

Overview The intent of this test is to ascertain a rough idea of your skills as a Senior .NET Developer. We will do this by evaluating your approach, prioritisation, communication, technical ability, and the implementation itself of the solution you submit.

This is not meant to be an exacting or laborious programming test. Instead, its aim is to allow you to express your opinions and preferences and to demonstrate your knowledge and technique. You should aim to showcase yourself and include as much detail as you see fit.

The requirements in this task are not always concrete, we'd like to assess your creativity and problem solving - if you make assumptions along the way, let us know what they were.

There is no right or wrong, no pass or fail. All options are valid as long as you explain them. Will you rely on any major 3rd party libraries? That's fine, explain your choice. In fact, this test is as much about communicating and making your intentions as well as the process clear, as it is about technical performance itself.

Delivery

- You should dedicate a maximum of 4 hours to complete as much as you can. Be aware we do not expect a fully completed solution at this time, but it should run.
 - Do your work within a git repository and develop the task in a similar manner as to how you would develop a task for your work.
 - Documentation of your thoughts and choices should be provided to go along with your solution - how you choose to provide this is up to you.
 - When you are finished, push your repo to Github etc. or zip it up and send it to us. You will need to send us an email with a link to the repo.
- Stack and Implementation Points**
- Use Git
 - Use any IDE of your choice
 - Use .NET 5
 - Use Entity Framework or similar • Data should be persisted somewhere, this can be in-memory or a real database, etc.
 - The solution should be properly layered, using appropriate patterns, abstractions and interfaces
 - The implementation should utilise DDD patterns

The Task

In order to manage scheduling within a doctor's practice, a calendar or schedule is necessary. You will build the backend API and services to manage the calendar/schedule, appointments and events around it. There is no need to build any frontend in this task.

Requirements

Must

- Attendees should comprise of a minimum of Name, Email Address, and whether they are Attending
- Events should comprise of a minimum of Title, Description, Attendees, a Start Time and End Time
- Sensible limits on field sizes should exist
- There should be notifications capability (Email/iCal/MQ/other) • Appropriate testing should be added

Should

- The API should implement the OpenAPI specification
- The API should have an auto-generated client that can be consumed by 3rd parties
- The API should have public facing auto-generated documentation

Could

- Because the preservation of data is of great importance and value, how could you deal with updates to such a system that occur on the same event?

API Functions

Must

- Create event
- Update event
- Delete / cancel event
- List calendar events (w/ filters) • Search for events

Should

- Accept/Reject event

Could

- You can add any other advanced features that you like o For example, checking attendee availability

Notes

- There won't be enough time to implement everything – scaffolding is quick and shows intention. Get as far as you can and show us clearly some architectural choices, a functioning vertical or a deep dive into parts of the system - whatever you think shows off your skills best
- We like tests, we expect you to show an appropriate level of understanding • We like an appropriate level and type of documentation
- We want to understand your thinking and process, commit messages are useful
- We expect you to choose an appropriate file structure and repository
- The code must compile and run
- We need to understand how we should use your solution